

Cómo guarda Git la historia por dentro

Objetos blob, tree y commit, referencias y ramas, y qué escribe un merge

Informe técnico. Las siglas en gris remiten a las referencias del final; las salidas de comandos son de un repositorio de prueba armado para este informe.

MATERIA	INTEGRANTES	DOCENTES	FECHA
Herramientas de desarrollo de software			Septiembre de 2026

NOMBRE DE UN OBJETO 40 caracteres: el SHA-1 del contenido PG §10.2	UNA RAMA EN DISCO 41 bytes 40 caracteres y un salto de línea PG §3.1	PADRES DE UN MERGE 2 las dos puntas que se combinan GM	BLOBS NUEVOS DEL MERGE 0 escribió 2 trees y el commit R
---	---	---	--

Resumen ejecutivo

Qué guarda. Por dentro, Git es un almacén de clave y valor: cada objeto se guarda en `.git/objects` bajo el hash SHA-1 de su contenido PG §10.2. Tres tipos arman la historia: el blob guarda el contenido de un archivo, el tree guarda un directorio (nombres, modos y hashes) y el commit guarda un tree raíz, sus padres, el autor y el mensaje.

Cómo se encadena. Cada commit es una foto completa del proyecto que nombra a su padre por el hash PG §3.1. Lo que no cambió no se copia: el tree nuevo apunta al mismo objeto. En la prueba, el segundo commit escribió 4 objetos y reutilizó 1 R.

Qué es una rama. Un archivo con el hash de un commit. HEAD normalmente no guarda un hash sino el nombre de la rama actual, y al hacer un commit la rama avanza sola PG §10.3.

Qué hace un merge. Si la rama actual es ancestro de la otra, Git solo avanza el puntero. Si divergieron, combina las dos puntas contra su ancestro común y escribe un commit con dos padres GM.

Tesis. Los objetos de Git no cambian nunca: un contenido distinto es otro objeto, con otro nombre. Lo único que se mueve son las referencias. Por eso una rama cuesta 41 bytes, y en la prueba un merge sin conflictos escribió solo dos trees y un commit.

EN ESTE DOCUMENTO

01 Git guarda cada contenido bajo el hash de ese contenido p.2 | **02** Un commit apunta a un tree raíz y al commit anterior p.2 | **03** Una rama es un archivo de 41 bytes con el hash de un commit p.3 | **04** Un merge mueve un puntero o escribe un commit con dos padres p.4 | Anexo: el segundo commit, paso a paso p.5 | Referencias p.5



Qué hace git commit: guarda los blobs de los archivos que cambiaron, escribe los trees y el commit, y mueve la rama actual al commit nuevo PG §10.2 · §3.1. El anexo lo repite con comandos de bajo nivel: → paso a paso, p. 5.

01 Git guarda cada contenido bajo el hash de ese contenido

Git es un sistema de archivos direccionable por contenido. `git hash-object -w` recibe un contenido, lo guarda en la base de objetos y devuelve su clave: un hash SHA-1 de 40 caracteres. Los dos primeros nombran la subcarpeta y los 38 restantes, el archivo PG §10.2.

```
$ find .git/objects -type f
$ echo 'harina, agua, sal' | git hash-object -w --stdin
23e2177961ce1f6375565a8f8bc7edfe00900232
$ find .git/objects -type f
.git/objects/23/e2177961ce1f6375565a8f8bc7edfe00900232
```

Un blob escrito a mano: la base de objetos empieza vacía y después tiene un archivo, en la subcarpeta de los dos primeros caracteres del hash R.

El hash no se calcula solo sobre el contenido. Git antepone un encabezado con el tipo, un espacio, el tamaño en bytes y un byte nulo; calcula el SHA-1 de todo eso y lo comprime con zlib antes de escribirlo PG §10.2. Ese objeto es un **blob**: guarda el contenido y nada más. El nombre del archivo no está en él.



De contenido a objeto. El hash sale del encabezado más el contenido, y ese hash, en azul, es a la vez el nombre del objeto y la ruta del archivo comprimido PG §10.2 · R.

```
$ git cat-file -t 23e2177
blob
$ git cat-file -s 23e2177
18
$ printf 'blob 18\000harina, agua, sal\n' | shasum -a 1
23e2177961ce1f6375565a8f8bc7edfe00900232 -
```

`cat-file -t` da el tipo y `-s`, el tamaño en bytes GC; la última línea recalcula el hash sin Git, con el encabezado delante, y da el mismo valor R.

La consecuencia es que el mismo contenido da siempre el mismo nombre. Dos archivos iguales, o el mismo archivo en dos commits, son un único objeto: en la prueba, el blob de `README.md` es el mismo en los dos primeros commits R.

02 Un commit apunta a un tree raíz y al commit anterior

El **tree** guarda lo que el blob no tiene. Cada entrada lleva un modo, un tipo, un hash y un nombre, y apunta a un blob o a otro tree, como un directorio de UNIX. Para archivos, los modos válidos son 100644 (normal), 100755 (ejecutable) y 120000 (enlace simbólico). El **commit** agrega el tree raíz, los padres, autor y committer con su marca de tiempo, una línea en blanco y el mensaje PG §10.2.

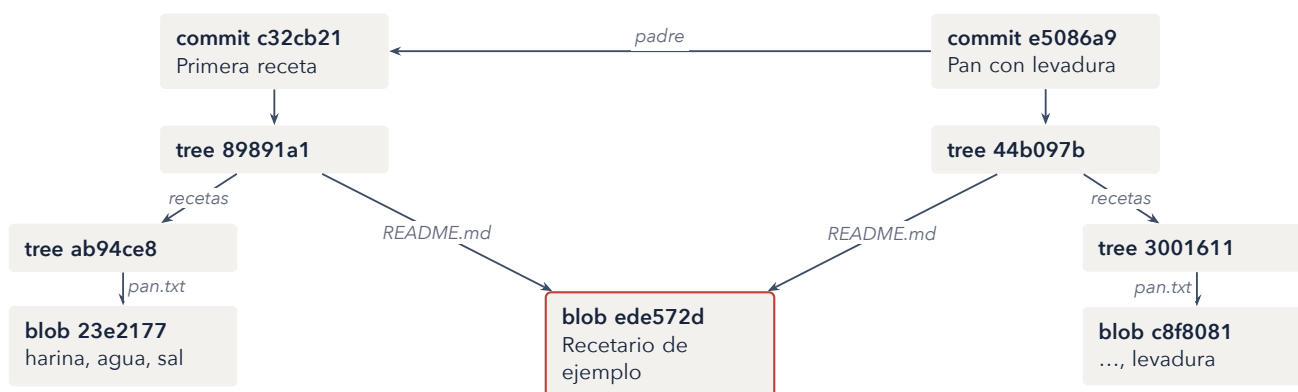
```
$ git cat-file -p HEAD
tree 89891a186edc343cc3c07c8f658664b8ca1b7c69
author Ana Ejemplo <ana@example.com> 1772445600 +0000
committer Ana Ejemplo <ana@example.com> 1772445600 +0000

Primera receta
$ git cat-file -p 'HEAD^{tree}'
100644 blob ede572d7bcbab9932e7b7e8a6c03c72863e97892 README.md
040000 tree ab94ce87fc1addce4753862b3eb9450927f52aab recetas
$ git cat-file -p 'HEAD^{tree}:recetas'
100644 blob 23e2177961ce1f6375565a8f8bc7edfe00900232 pan.txt
```

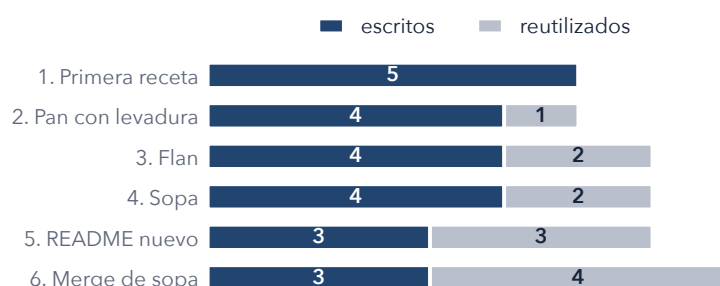
```
$ git cat-file -p HEAD | head -3
tree 44b097ba92a4fa7036d9f33354a5c389797e4341
parent c32cb211a8a6e7b0aca88b2c192866aeb5edb84d
author Ana Ejemplo <ana@example.com> 1772532000 +0000
```

El primer commit de la prueba y sus dos trees; después, el comienzo del segundo commit, que ya nombra a su padre R.

En el segundo commit solo cambió `pan.txt`. Git escribió cuatro objetos (el blob nuevo, los dos trees que lo contienen y el commit) y reutilizó el blob de `README.md`. Así Git guarda fotos completas y no diferencias PG §3.1 sin copiar el proyecto entero en cada commit.



Los dos primeros commits de la prueba. Los nombres de archivo van en las flechas porque los guarda el tree, no el blob. Con borde terracota, el blob de `README.md`: no cambió, y los dos trees raíz apuntan al mismo objeto R.



Después del primero, ningún commit de la prueba copia el proyecto entero: cada uno escribe 3 o 4 objetos y reutiliza el resto. «Escritos» cuenta el propio commit R.

En los seis commits de la prueba, lo reutilizado crece con el proyecto, y lo escrito se mantiene en 3 o 4 objetos después del primero. El merge, el último, no escribió ningún blob: todo su contenido ya existía R.

Los objetos sueltos son la forma inicial. Cuando se acumulan, o con `git gc`, Git los empaqueta en un packfile y guarda algunas versiones como diferencias contra otra PG §10.4.

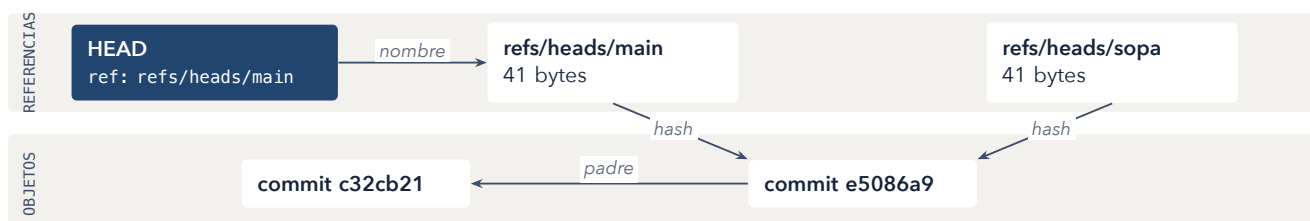
03 Una rama es un archivo de 41 bytes con el hash de un commit

Recordar hashes no es práctico, así que Git guarda nombres simples, las **referencias**, en `.git/refs` PG §10.3. Una rama es una referencia en `refs/heads`: un archivo con los 40 caracteres del hash de un commit y un salto de línea PG §3.1. Crear una rama es escribir ese archivo: `git branch sopa` copió el hash al que apuntaba `main` R.

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
e5086a9f872b9f4e862a978f5a254cdc49af732c
$ wc -c < .git/refs/heads/main
41
$ git branch sopa
$ cat .git/refs/heads/sopa
e5086a9f872b9f4e862a978f5a254cdc49af732c
```

Las referencias de la prueba después del segundo commit: HEAD guarda un nombre; `main` y la rama nueva, el mismo hash R.

HEAD es una **referencia simbólica**: en lugar de un hash contiene `ref: refs/heads/main`, la rama actual. Al hacer un commit, Git toma como padre el commit al que apunta HEAD PG §10.3, y la rama avanza sola al commit nuevo PG §3.1. Si HEAD guarda un hash en vez de un nombre, el repositorio queda en estado *detached HEAD* PG §10.3.



Arriba, archivos que se reescriben con cada commit o rama nueva; abajo, objetos que no cambian. En azul, HEAD, que nombra una rama PG §10.3 · R.

Las etiquetas también son referencias. Una liviana es una referencia que no se mueve; una anotada apunta a un cuarto tipo de objeto, con quien la creó, la fecha, un mensaje y un puntero. Las referencias remotas, en `refs/remotes`, son de solo lectura: guardan dónde estaba cada rama del servidor la última vez que hubo contacto PG §10.3.

04 Un merge mueve un puntero o escribe un commit con dos padres

Si la rama actual es ancestro del commit que se integra, no hace falta un commit nuevo: Git mueve `HEAD` y la rama a ese commit. Es el **avance rápido** (*fast-forward*), y `--no-ff` lo evita GM. En la prueba, `postres` estaba un commit delante de `main`, y el merge solo movió `main`.

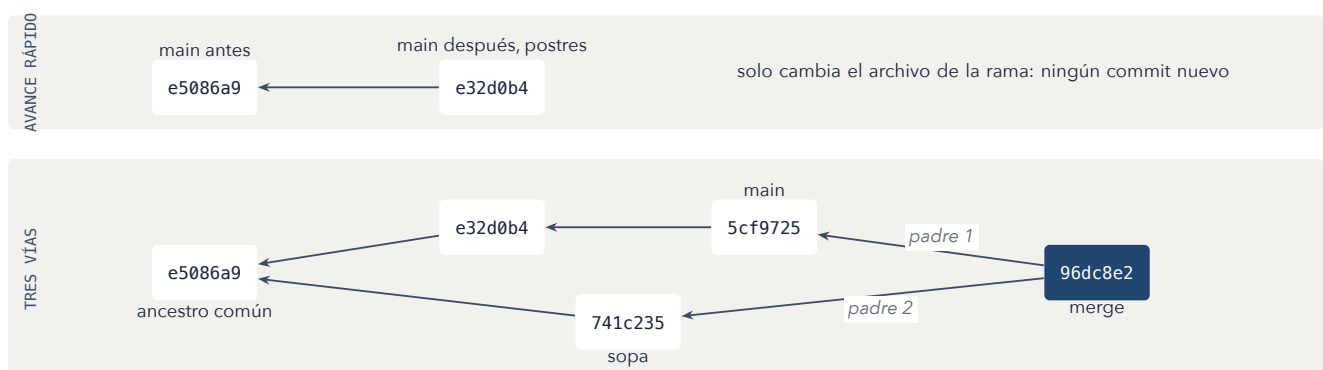
Si las ramas divergieron, Git hace un **merge de tres vías** con las dos puntas y su mejor ancestro común, el que da `git merge-base` GB. El resultado es un commit de merge con las dos puntas como padres; al integrar una sola rama, la estrategia por defecto es `ort` GM.

```
$ git merge postres
Updating e5086a9..e32d0b4
Fast-forward
 recetas/flan.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 recetas/flan.txt
```

```
$ git merge-base main sopa
e5086a9f872b9f4e862a978f5a254cdc49af732c
$ git merge --no-edit sopa
Merge made by the 'ort' strategy.
 recetas/sopa.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 recetas/sopa.txt
$ git cat-file -p HEAD
tree e5fb560fb2e91ea96101922f271ed707eb450a1a
parent 5cf9725703f75829e73fbd83b6c31b210f5a9cd0
parent 741c235c93ffe14dc65837348fd10bd7cac47192
author Ana Ejemplo <ana@example.com> 1772877600 +0000
committer Ana Ejemplo <ana@example.com> 1772877600 +0000

Merge branch 'sopa'
```

Los dos casos en la prueba: el avance rápido no escribe ningún commit; el de tres vías parte del ancestro común y escribe uno con dos líneas parent R.



Arriba, `main` salta de `e5086a9` a `e32d0b4` sin objetos nuevos. Abajo, el commit de merge, en azul, tiene dos padres, y el ancestro común es el mismo `e5086a9`. Las flechas van de cada commit a su padre R.

Si un archivo no se puede combinar solo, `HEAD` queda donde estaba, `MERGE_HEAD` apunta a la otra punta y el índice guarda hasta tres versiones: la del ancestro, la de `HEAD` y la de la otra rama. El archivo queda con marcadores de conflicto, y `git merge --abort` vuelve al estado anterior GM.

Anexo: el segundo commit, paso a paso

`git commit` se puede reproducir con comandos de bajo nivel que hacen lo mismo: `git update-index` pone el archivo en el índice, `git write-tree` escribe el tree, `git commit-tree` escribe el commit con su padre y `git update-ref` mueve la rama PG §10.2 · §10.3. En una copia de la prueba, con el mismo autor, la misma fecha y el mismo mensaje, los cuatro pasos dan exactamente el tree `44b097b` y el commit `e5086a9` que había escrito `git commit R`.

```
$ git update-index recetas/pan.txt
$ git write-tree
44b097ba92a4fa7036d9f33354a5c389797e4341
$ echo 'Pan con levadura' | git commit-tree 44b097b -p HEAD
e5086a9f872b9f4e862a978f5a254cdc49af732c
$ git update-ref refs/heads/main e5086a9
$ git log --oneline
e5086a9 Pan con levadura
c32cb21 Primera receta
```

El segundo commit de la prueba, hecho a mano: los hashes coinciden con los del commit hecho con `git commit`, porque el contenido de cada objeto es el mismo R.

Referencias

- [PG] Chacon, S. y Straub, B. *Pro Git*, 2.^a edición. Secciones 3.1 «Branches in a Nutshell», 10.2 «Git Objects», 10.3 «Git References» y 10.4 «Packfiles». Licencia CC BY-NC-SA 3.0. <https://git-scm.com/book/en/v2>.
- [GM] Documentación oficial de Git, `git-merge`: <https://git-scm.com/docs/git-merge>.
- [GB] Documentación oficial de Git, `git-merge-base`: <https://git-scm.com/docs/git-merge-base>.
- [GC] Documentación oficial de Git, `git-cat-file`: <https://git-scm.com/docs/git-cat-file>.
- [R] Repositorio de prueba de este informe, armado con `demo.sh` (junto al documento) y Git 2.50.1. Autor, fechas y contenido son fijos, así que los hashes se reproducen.